

Chapter 3

Bitcoin

Bitcoin & Blockchain

- Concept was invented by a person (or group of people) using the name Satoshi Nakamoto in 2008.
- To serve as the public transaction ledger of the cryptocurrency bitcoin.
- The identity of Satoshi Nakamoto is unknown.
- Some says its, **Samsung, Toshiba, Nakamichi, and Motorola.** Taken together, the first two to four letters of these companies spell out Satoshi Nakamoto.
- Nakamoto stated that work on the writing of the code for bitcoin began in 2007.
- On 18 August 2008, s/he or a colleague registered the domain name bitcoin.org and created a web site at that address.

How bitcoin works?

Bitcoin is a combination of a

- Currency + Payment system + Collection of algorithms
- Hardware and software implementations
- The goal of bitcoin is to enable payments with low transaction costs
- Rewards and fees per transaction: <https://blockchain.info/stats>

Bitcoin

- **Bitcoin** has introduced Proof of Work model for mining:
 - To verify the legitimacy of a transaction
 - To avoid double spending
 - To create new digital currencies by rewarding miners
 - It works as per the Byzantine Fault Tolerance approach
 - https://www.youtube.com/watch?v=kZXXDp0_R-w

Bitcoin

- Satoshi Nakamoto's proposes Blockchain in 2008
- Blockchain is considered a type of payment rail.
- Private blockchains have been proposed for business use.
- The bitcoin blockchain file size reached(gigabytes):
 - In August 2014 ----- 20 GB
 - In January 2015 ---- 30 GB
 - from January 2016 to January 2017 ----grew from 50 GB to 100 GB in size.
- In 2021 to 23 ---- 331.07 to 400 GB
- On January 29, 2025 it is **633.77 gigabytes**

How Many Bitcoins Will Ever be Created?

- The maximum and total amount of bitcoins that can ever exist is 21 million.

How Many Bitcoins Are Left to Be Mined?

- 19.9 million bitcoins have been mined, leaving about 1.1 million bitcoins remaining to be mined.
- However, you can always buy bitcoins from existing users on exchanges.

- **How Many Bitcoins Are Lost?**

- There's no exact answer. One recent estimate is that about 3-4 million bitcoins are lost forever.

Getting started

- Download software to create a Bitcoin wallet (<https://bitcoin.org/en/choose-your-wallet>)
- The wallet holds the private keys you use to prove your own specific Bitcoins.
- The software creates public/private key pairs for you as needed.
- The wallet also contains software that allows you to send and receive bitcoins.

Influencing factors

May influence the supply and demand of Bitcoins price:

- Confidence of people in Bitcoin and fear of uncertainty
- Press coverage with good and bad news on Bitcoin
- Some people own Bitcoins and do not allow them to flow through the market
- Some people keep buying and selling to minimize risk.
- This is why the liquidity level of Bitcoin keeps on changing.
- Acceptance of Bitcoins by big e-commerce giants
- Banning of Bitcoins in specific countries

Blockchain Digital Wallets



Desktop wallets



Mobile wallets



Cold wallets/
Hot wallets



Online web wallets



Paper wallets

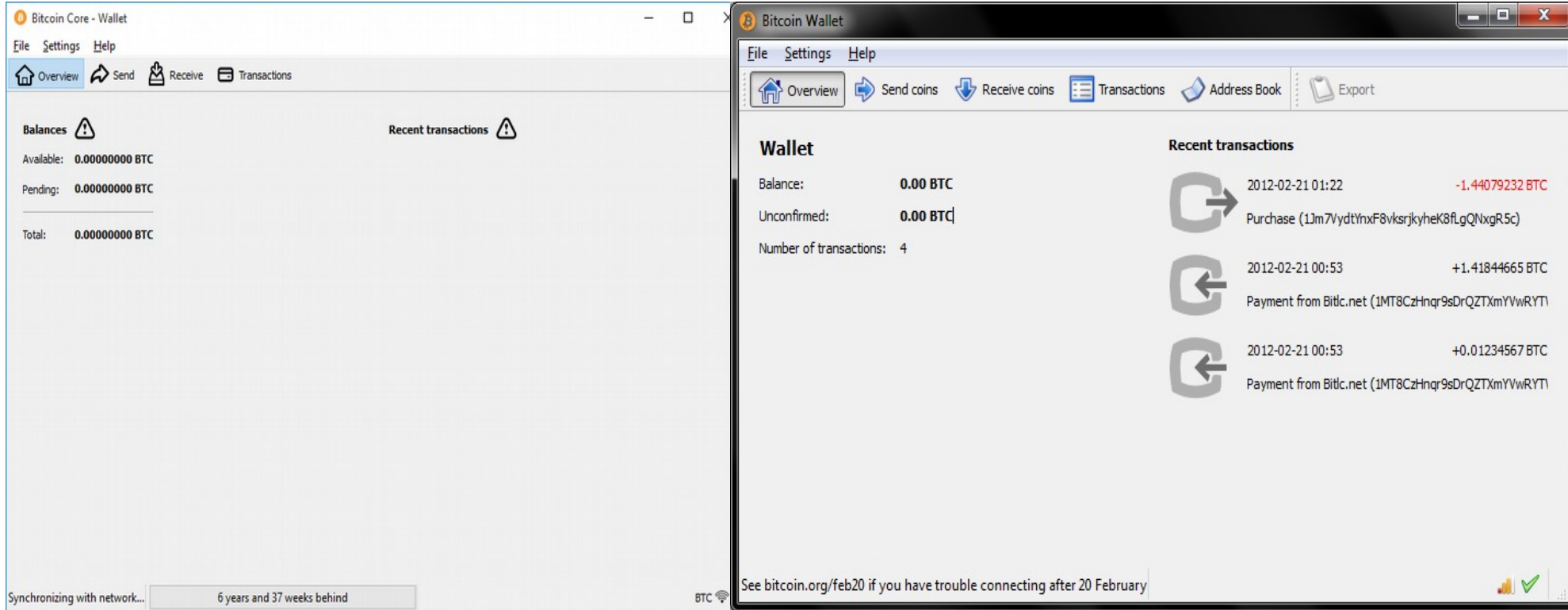


Hardware wallets



Bitcoin wallets

Bitcoin Core (original) wallet on first start-up



Varieties of Bitcoin wallets

- Mobile wallets
- Desktop wallets
- Browser-based web wallets
- Hardware wallets etc.

with varying levels of security

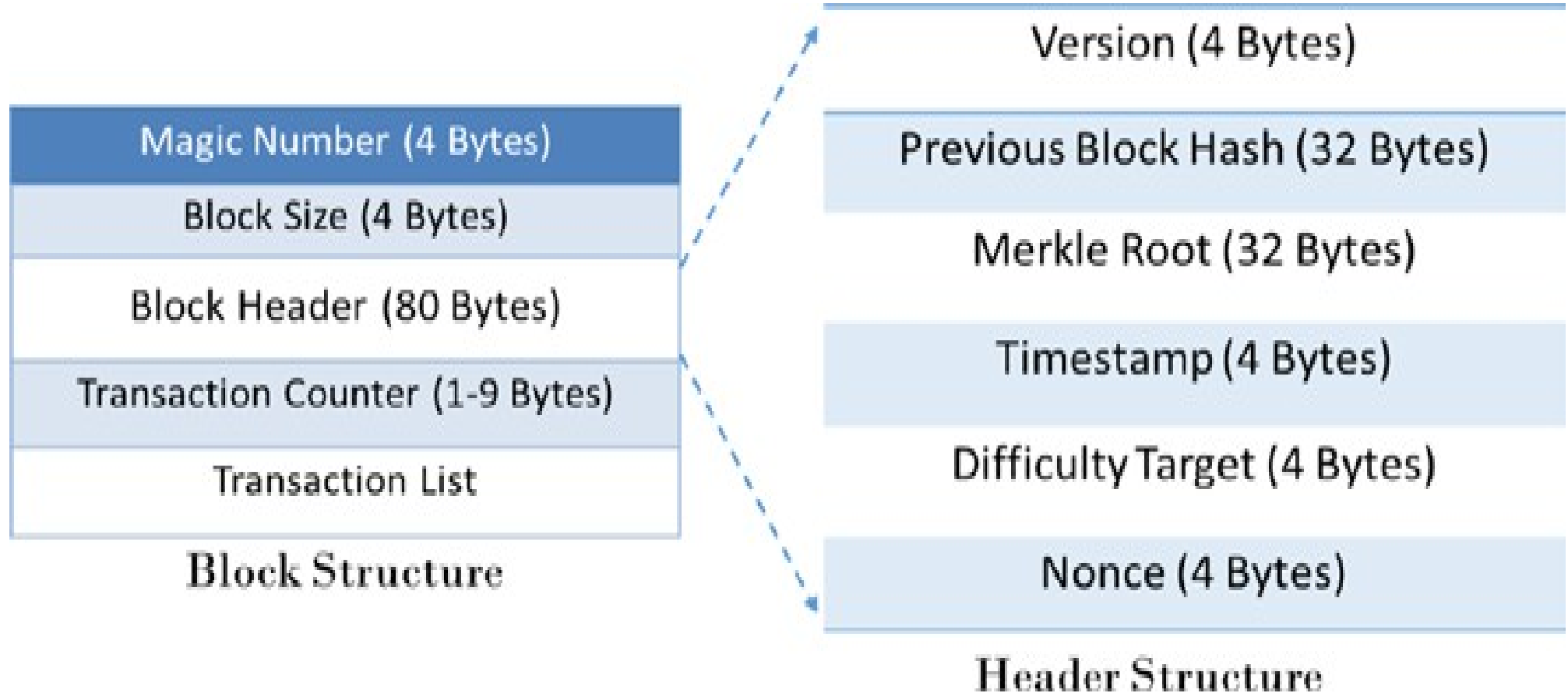
Security in Wallets

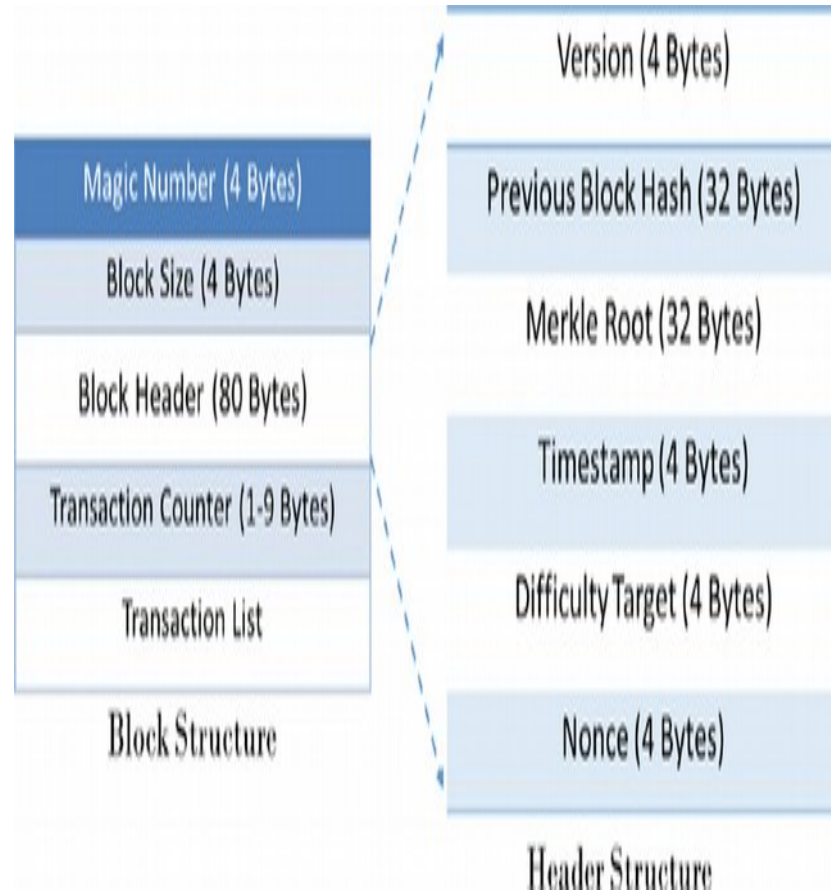
- Different wallet types have different levels of security
- Many online wallet services suffered from security breaches.
- Must practice **two-factor authentication** :
 - **use small amounts in your wallets** and keep the remainder separately in a safe environment.
 - **e.g: An offline wallet or cold wallet** that is not connected to the network
 - There should be proper backup mechanisms in case you lose your computer/mobile.

The Bitcoin Blockchain: DB

- The Bitcoin Core client uses the
 - **LevelDB database of Google** to store the blockchain data structure internally
- Each block is identified by its hash (using SHA256 algorithm).
- Every block contains the hash of the previous block in its header section.
- **It is the hash of entire block including header.**

Block Structure





version	02000000
previous block hash (reversed)	17975b97c18ed1f7e255adf297599b55 330edab87803c8170100000000000000
Merkle root (reversed)	8a97295a2747b4f1a0b3948df3990344 c0e19fa6b2b92b3a19c8e6badc141787
timestamp	358b0553
bits	535f0119
nonce	48750833
transaction count	63
coinbase transaction	
transaction	
...	

Block hash

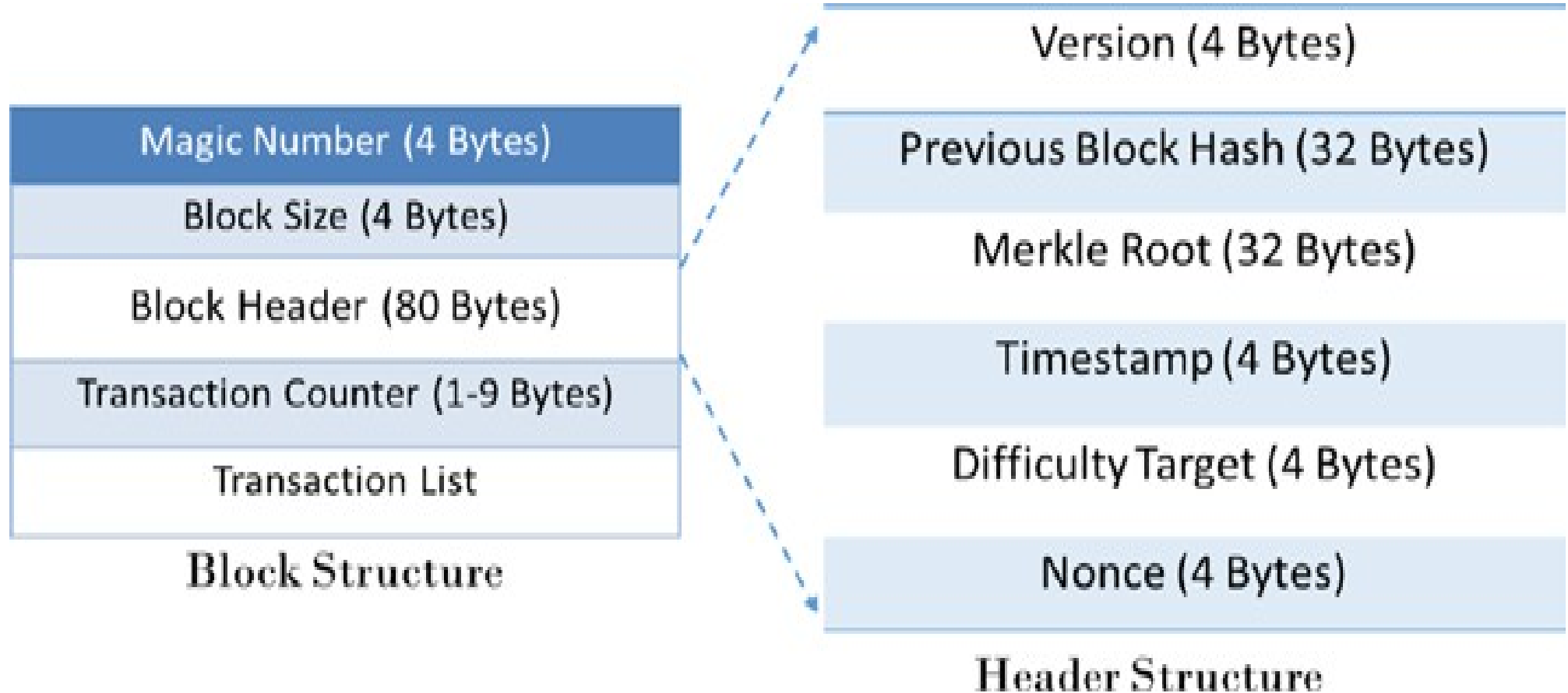
0000000000000000
e067a478024addfe
cdc93628978aa52d
91fabd4292982a50

Block Structure

- Magic Number : 4 bytes
 - It has a fixed value 0xD9B4BEF9,
 - which indicates the start of the block
- Block Size : 4 bytes
 - This indicates the size of the block.
 - Original Bitcoin blocks are of 1MB
 - Newer version of Bitcoin called “Bitcoin Cash” is of 2MB.
- Block Header: 80 bytes
 - It comprises of: Previous Block’s hash
 - Nonce, Merkle Root etc

- Transaction Counter 1–9 bytes (variable length)
 - There is a variable number of transactions in every block.
 - It indicates total number of transactions included in the block.
 - Transaction may be of different size.
- Transaction List --> Variable in number, but fixed in size
 - Lists of all the transactions in a given block.
 - Depending on block size (whether 1MB or 2MB)

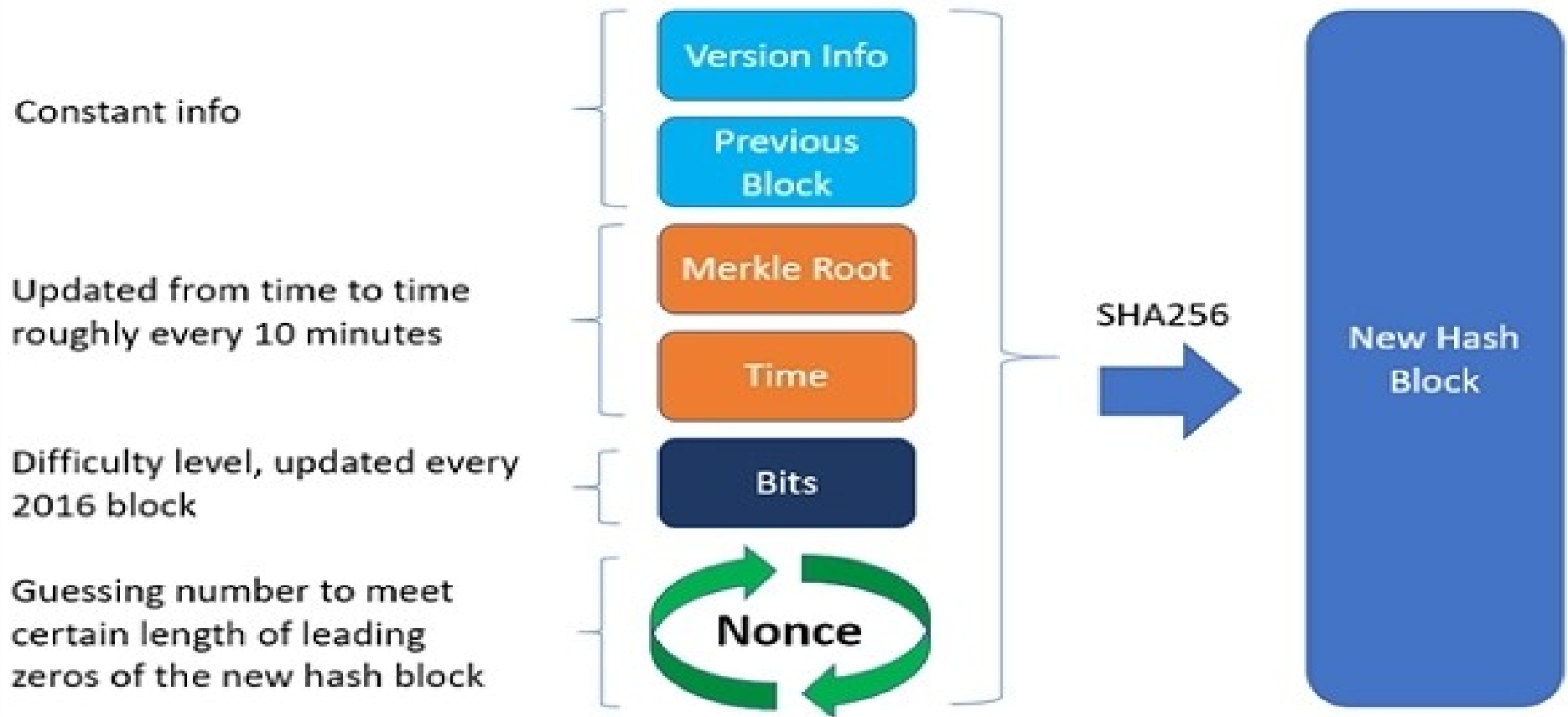
Block Structure



Block Header Components

- **Version** **4 bytes**
 - Version number of Bitcoin protocol
 - Ideally each node running Bitcoin protocol → same version number
 - **As of February 5, 2025: Bitcoin Core is 28.1**
- **Previous Block Hash** **32 bytes**
 - It contains the hash of the previous block
 - It hashed with SHA256 algorithm
 - SHA256 produces:**
 - 256-bit (32-byte) hash value, which is a hexadecimal number, 64 digits long
 - 4e3b7f2a1c5d9e8f0b6a4c3d7e9f1d2a5c6b8e0f9a2d3c4b7e8f1a2c3d5b6a7

Bitcoin Block Hashing



Timestamp, Nonce and Difficulty Target

- A unique hash value needs to be generated by miners for a new mined block
- They use 32 bit nonce value to create unique hash
- **Timestamp:** is a Unix time stamp (started from 1st January 1970)
- As timestamp value is changed at every next second, the combination of nonce range and timestamp can be identified unique
- When a miner creates a new block, the timestamp is set to the current network time or slightly adjusted to fit within acceptable limits.
- It must be greater than the median of the previous 11 blocks' timestamps and cannot be more than two hours into the future to prevent manipulation.

- **Nonce**("number only used once") **4 bytes**
 - The nonce in the header **starts from 0 (zero)**.
 - Is a 32-bit (4-byte) field whose value is adjusted by miners
 - Nonce is changed to get particular **unique** hash value
 - so that the hash of the block will be **less than or equal to** the current (**difficulty target**) target of the network.
 - This is the random number that satisfied the PoW puzzle during mining.
 - The miner has to keep on increment this nonce and hashing the header till the hashvalue is less than the target.
 - <https://demoblockchain.org/block>

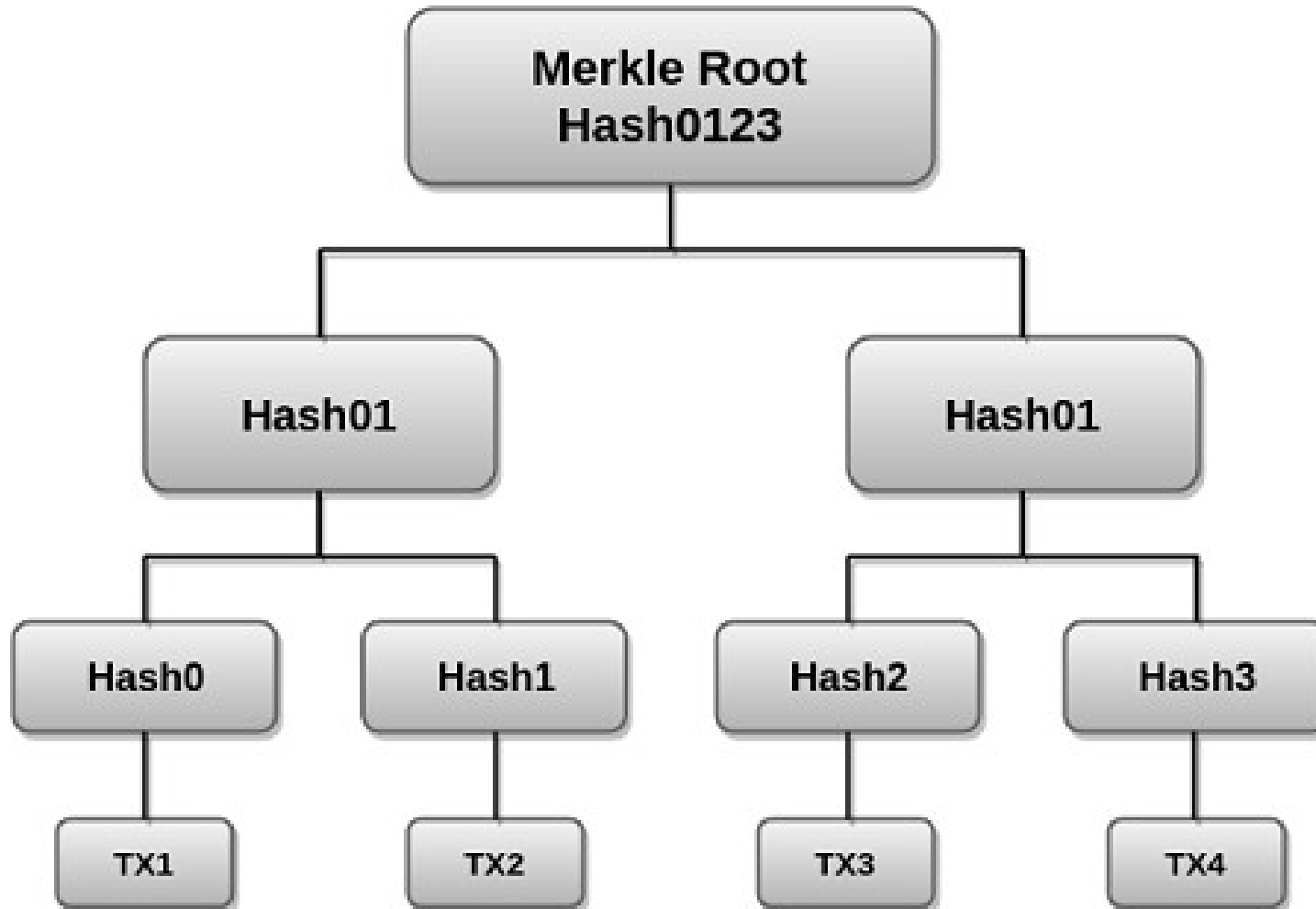
- **Difficulty Target** **4 bytes**

- It is the proof-of-work that was set for this block when it was mined
- Bitcoin is designed such that every **2,016 blocks should take two weeks to be generated**
- After every 2,016 blocks, all the nodes compute → new difficulty target value
- Mathematically, it would be around **ten minutes for every block**
- As of February 7, 2025, the Bitcoin network's mining difficulty is approximately 108.11 trillion (T).
- The latest data is indicating that 1,606 blocks have been mined so far in this cycle.
- This represents approximately 79.66% of the total blocks for this period.
- There are 410 blocks remaining until the next difficulty adjustment, which is estimated to occur on February 9, 2025.

Difficulty Target

- The Bitcoin network has a **global block difficulty**.
- Valid blocks must have a hash below this target.
- Difficulty is changed every 2016 blocks based on the time it took to discover 2016 previous blocks.
- If previous 2016 blocks were found in more than two weeks
 - The mining difficulty will be lowered
 - if they were mined faster then that it will be raised.

Merkle Root

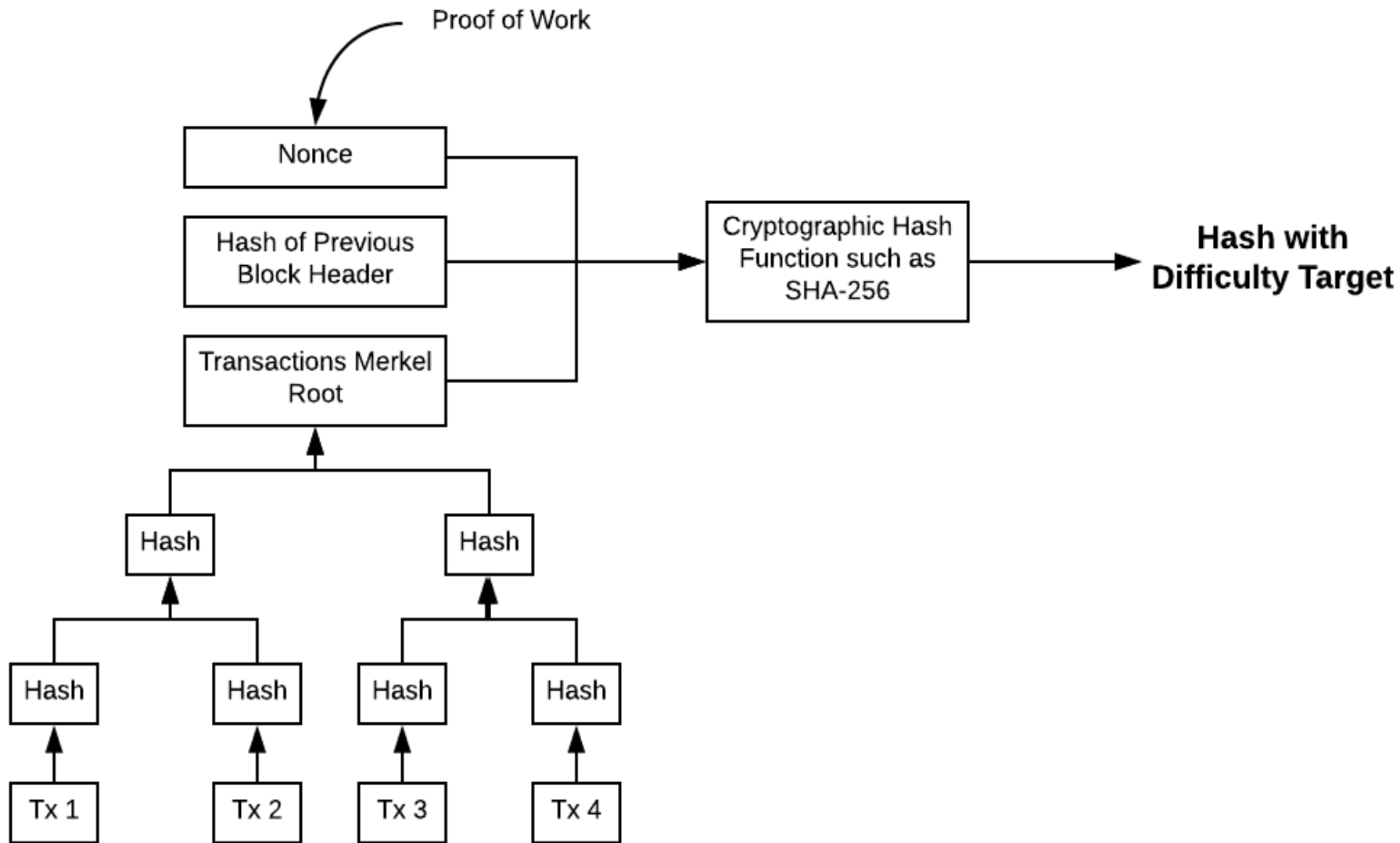


Merkle Root

- The root hash of a Merkle tree, which is a binary tree structure used to summarize all transactions in a Bitcoin block.
- Helps in quick verification of whether a transaction is included in a block without downloading the full block.
- How it works?
 - Each transaction's hash (TXID) is taken.
 - Pairs of hashes are repeatedly hashed together until a single Merkle root is formed.
 - This root is stored in the block header and is essential for proof-of-work verification.
 - It is included in every Bitcoin block header.

Merkle Root

- Merkle roots are used in cryptocurrency to make sure data blocks passed between peers on a peer-to-peer network are :
 - whole, undamaged, and unaltered
- It is used in SPV (Simplified Payment Verification) wallets for lightweight verification of transactions.



Bitcoin Network

Bitcoin Peer-to-Peer Network

- Bitcoin communications are decentralized.
- The P2P network is composed of different types of nodes with varying roles and functionalities.
- Network topology is random.
- Transaction and block records are broadcasted to all nodes

The Bitcoin Nodes in n/w

- Every node is
 - **equal in terms of authority** and has a flat structure
 - but the nodes can be full nodes or lightweight nodes
- Full nodes:
 - can do almost every permissible activity in the Bitcoin
 - such as mining transactions and broadcasting transactions
 - can **provide wallet services**

Full node

- All the node must stay permanently connected to the Bitcoin network and here all transactions taking place.
- For being a full node: It is important that you have
 - A good network connection
 - A good storage (at least 200GB)
 - At least 2GB RAM dedicated to it
- This requirement may further change and require more resources with time

Other nodes

Relay Nodes

- Do not mine but help propagate transactions and blocks efficiently.
- Often run by businesses or organizations to improve network connectivity.

Masternodes (Not in Bitcoin, but relevant in other networks)

- Some blockchains like Dash use masternodes for governance and faster transactions.

Light weight nodes - thin clients

- Help in verifying the transactions for the full nodes.
- Majority of nodes in the Bitcoin network are SPVs.
- Cannot mine new blocks but
 - can verify transactions by using **Simplified Payment Verification** (SPV)
- Can participate in pool mining
 - where there are many nodes trying to mine new blocks together

Simplified Payment Verification (SPV)

- SPV lets you validate your transactions without having to worry about anybody else's transactions.
- **SPV wallet is the same as a type of bitcoin wallets**, ensures your security and your data in active chain.
- **SPVs cannot validate transactions**, when they don't have all the
 - unspent transaction outputs (UTXOs) except for their own.

A UTXO is an unspent transaction output

- User/transactions can use existing UTXO to create new UTXO
- Received amounts (UTXOs) are used individually during a transaction and new outputs are created
 - one for the receiver and,
 - One for the amount that is left over

Let's take a look at a simplified example of how the UTXO model works in Bitcoin transactions:

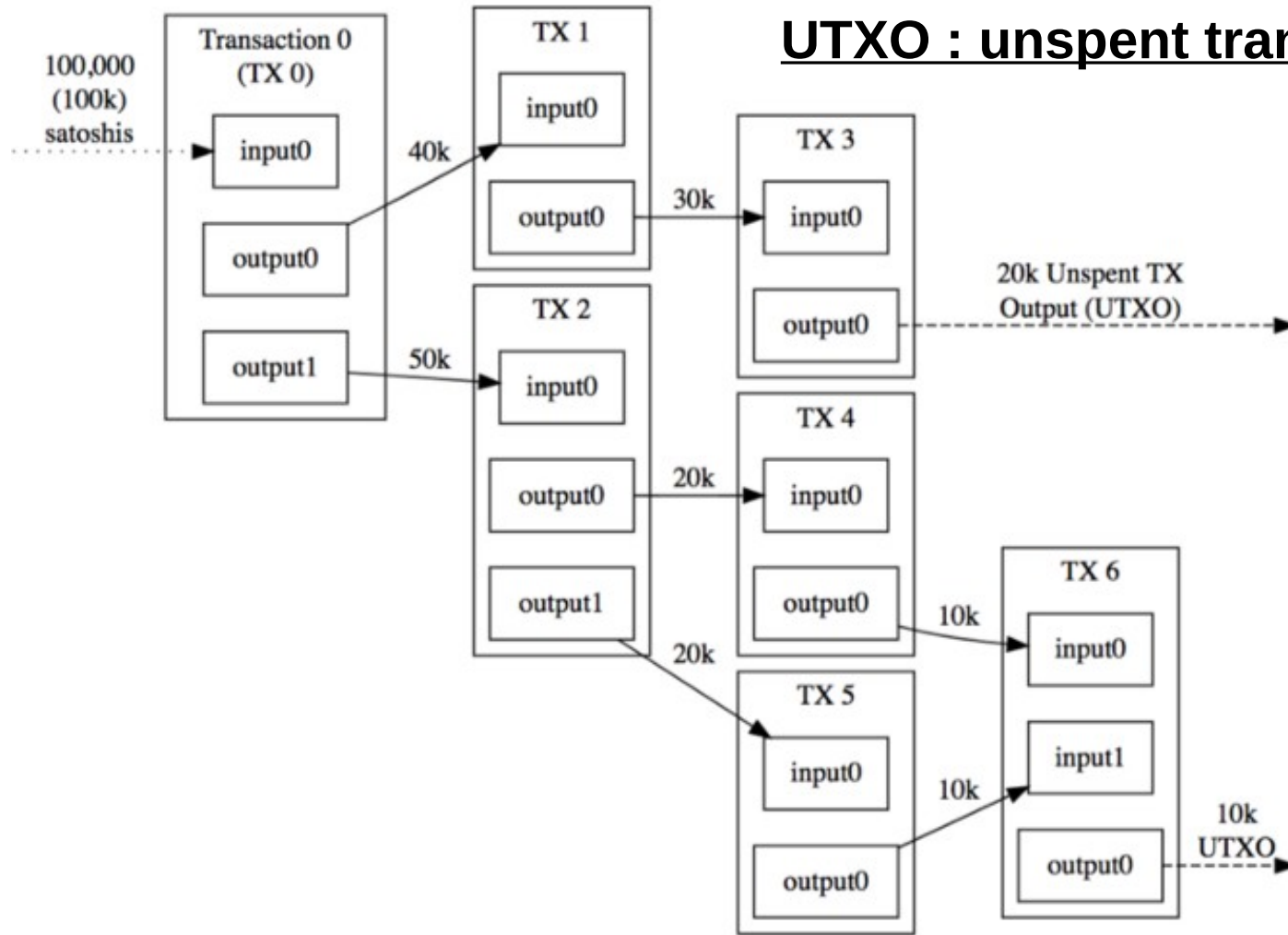
1. Alice gains 12.5 bitcoins through mining. Alice's wallet is associated with one UTXO record of 12.5 bitcoins.

2. Alice wants to give Bob 1 bitcoin. Alice's wallet first unlocks her UTXO of 12.5 bitcoins and uses this whole 12.5 bitcoins as input to the transaction. This transaction sends 1 bitcoin to Bob's address and the remainder of 11.5 bitcoins is sent back to Alice in the form of a new UTXO to a newly-created address (owned by Alice).

3. Say there was another UTXO of 2 bitcoins associated with Bob prior to step 2, Bob's wallet now shows that his balance is 3 bitcoins. Bob's wallet now keeps track of two UTXOs: one from before and another from the transaction in step 2. Each UTXO needs to be unlocked if Bob wishes to spend them.

The Account/Balance Model, on the other hand, keeps track of the balance of each account as a global state. The balance of an account is checked to make sure it is larger than or equal to the spending transaction amount.

UTXO : unspent transaction output



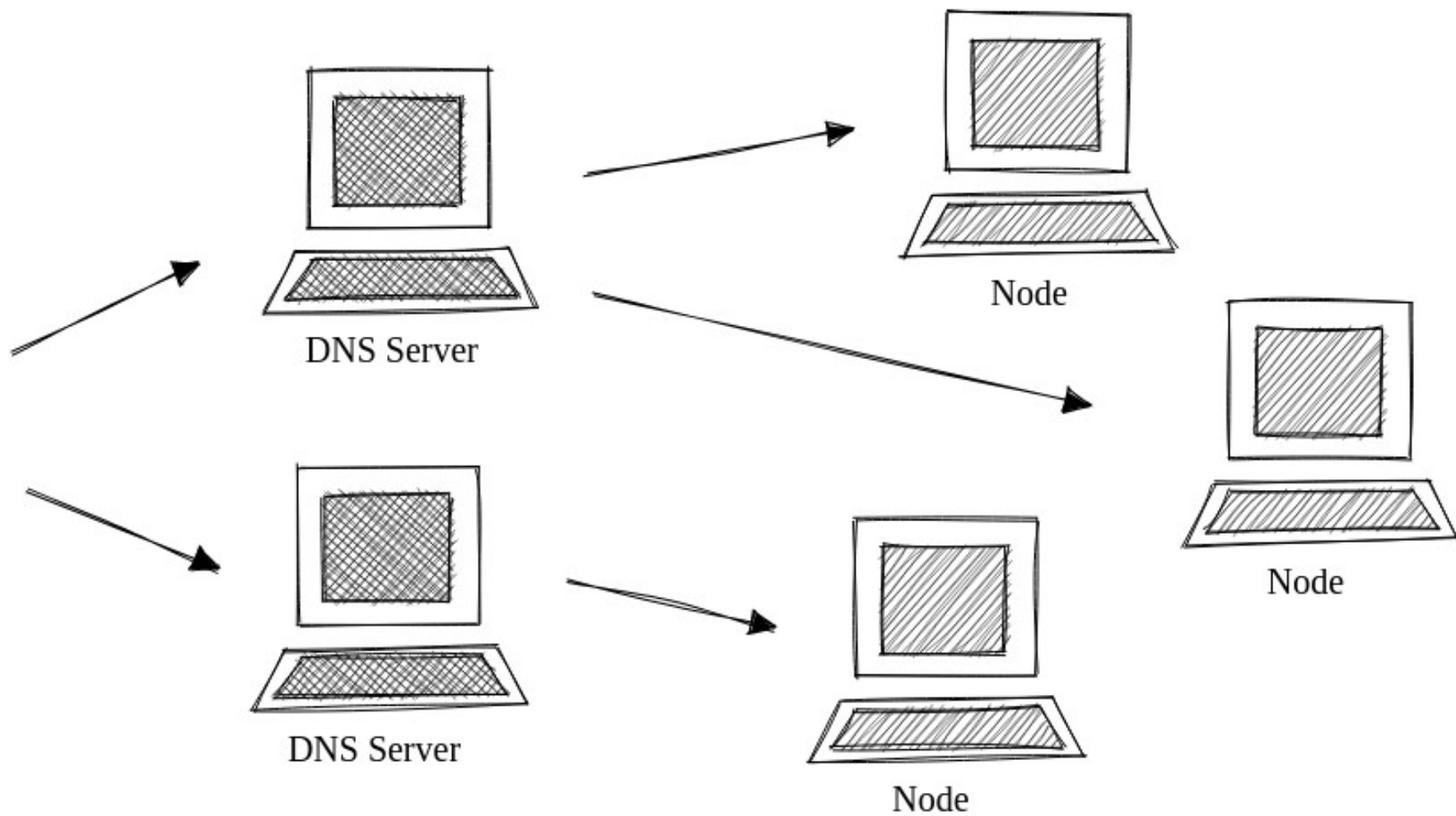
Triple-Entry Bookkeeping (Transaction-To-Transaction Payments) As Used By Bitcoin

Bitcoin Network Communication between peers

Network Discovery for a New Node

- When **started first time**, a program does not have the IP address of any full node. So, they used several methods to find the peers.
- **One of them is DNS seeds**
 - DNS seeds are servers which contains lists of IP addresses.
 - A **DNS server which returns IP addresses of full nodes** on the Bitcoin network to assist in peer discovery.
 - Several DNS seeds are hardcoded in them
 - DNS seeds are maintained by Bitcoin community members

Bitcoin
Source
Code

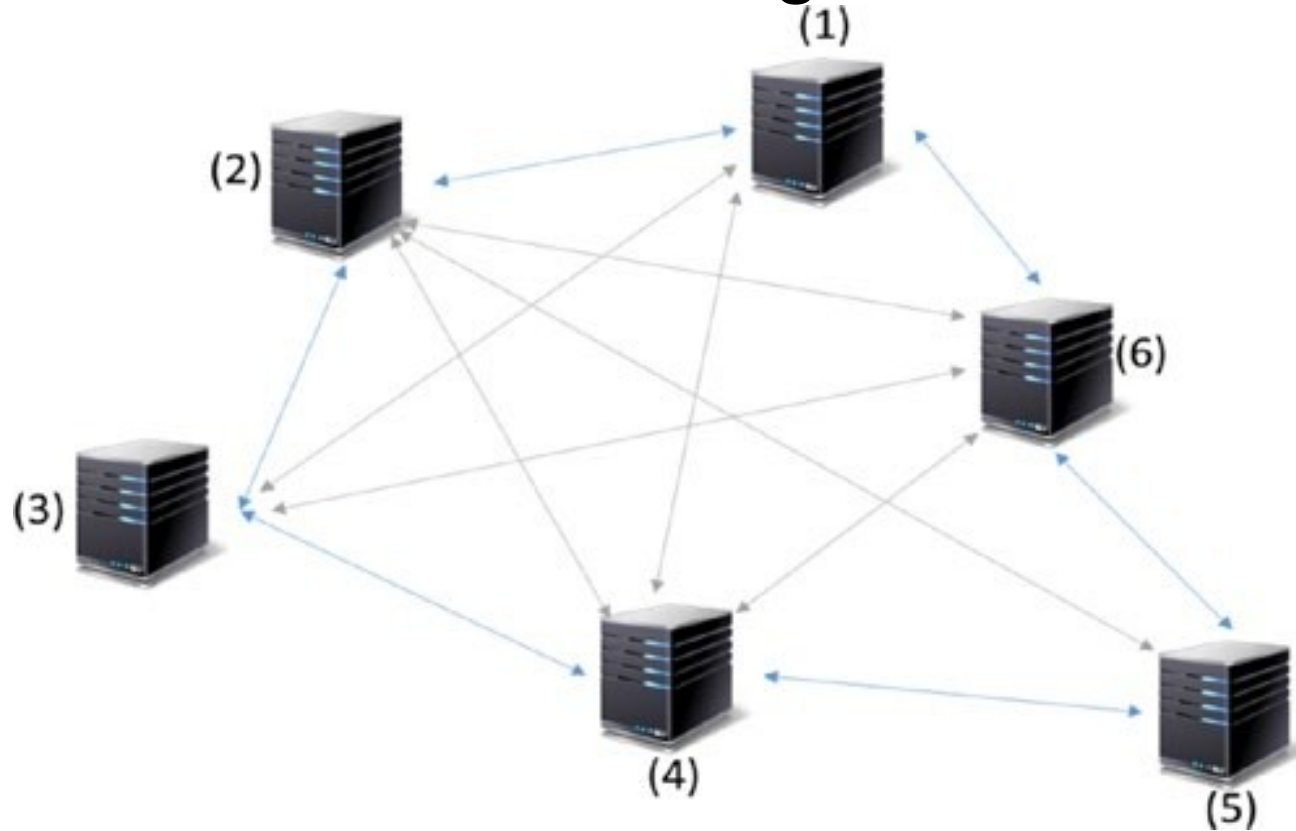


Network Discovery for a New Node

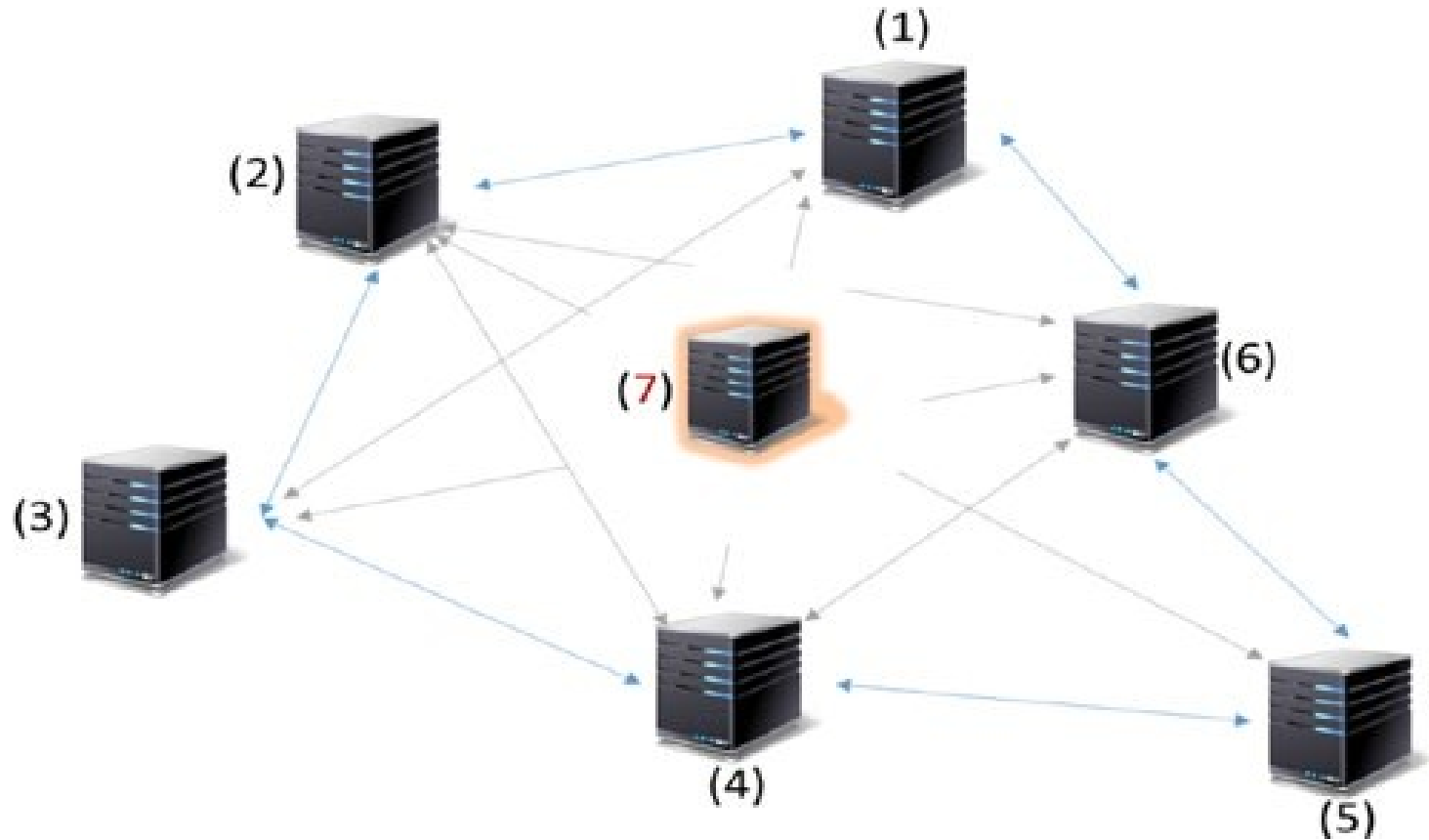
- **Some community members provide**
 - Static DNS seeds by manually entering the IP addresses and port numbers
 - Dynamic DNS seed servers that can automatically get the IP addresses of active Bitcoin nodes
 - The clients (Bitcoin Core) also maintain a hardcoded list of IP addresses : **stable Bitcoin nodes.**
 - Such nodes are **called bootstrap nodes**
 - whose endpoints are already available with the source code itself.

Network Discovery for a New Node

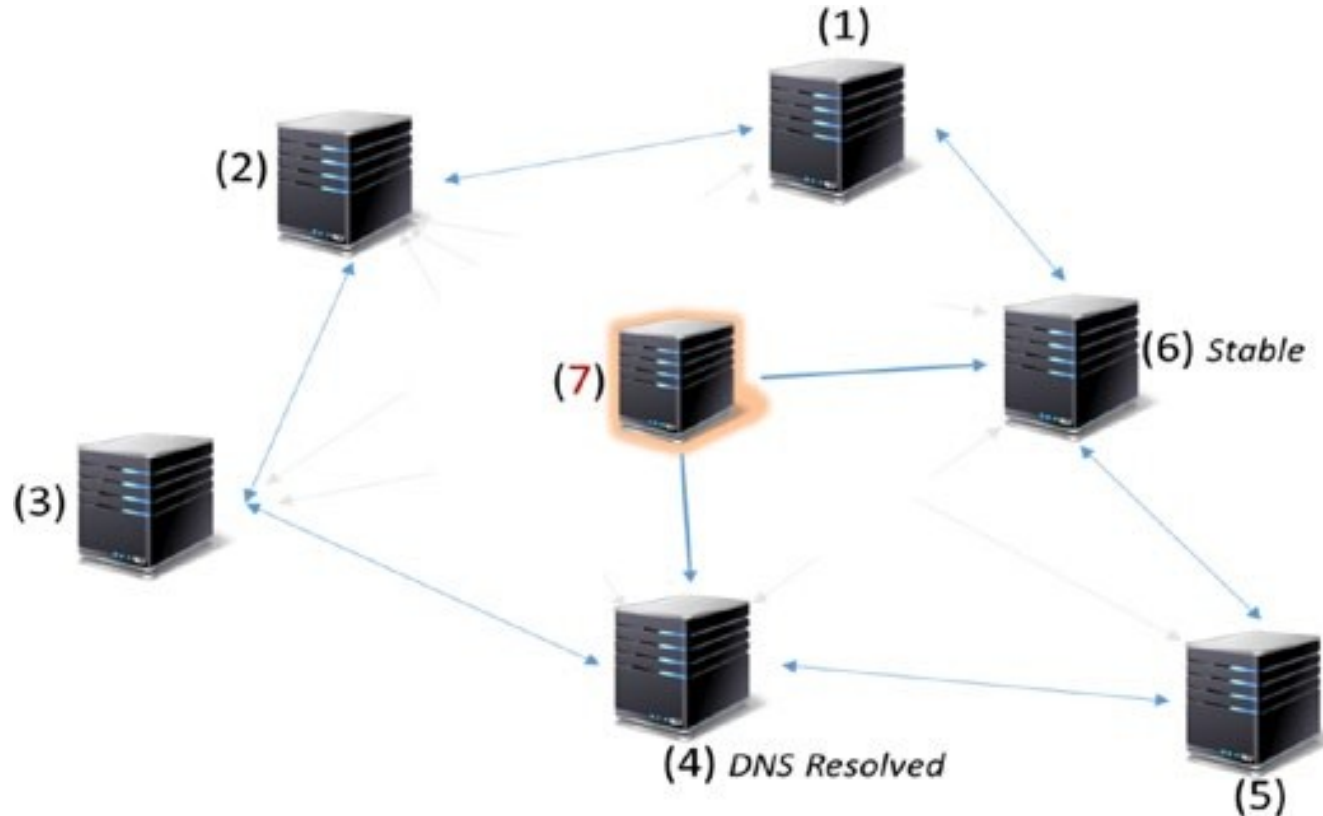
- Step 1: Bitcoin network in general



- Step-2: A new node 7th trying to join the network

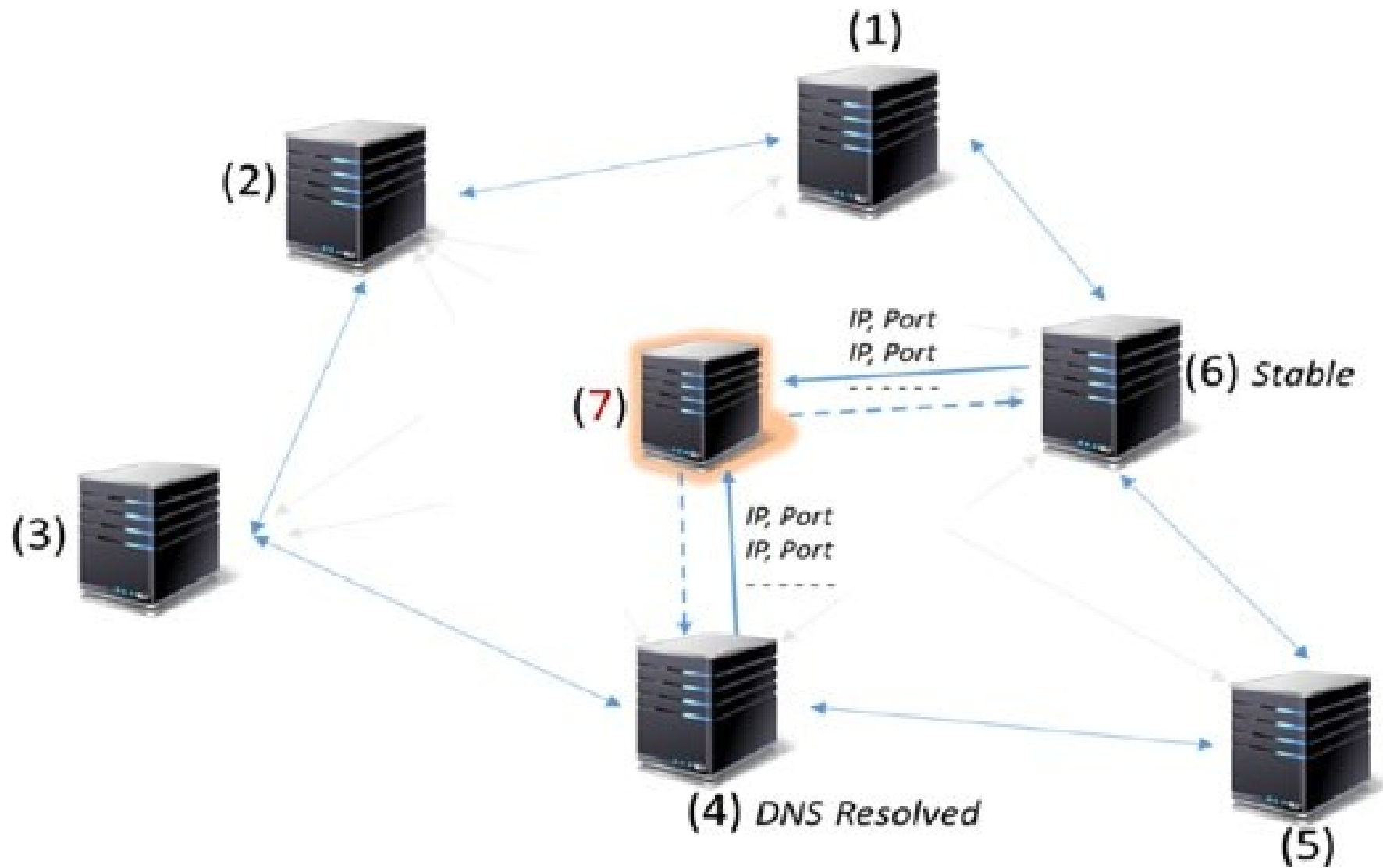


- **Step-3:** The 7th node will try to reach out to as many nodes as it can either **using DNS seeds** or using the **list of stable Bitcoin** nodes in the list

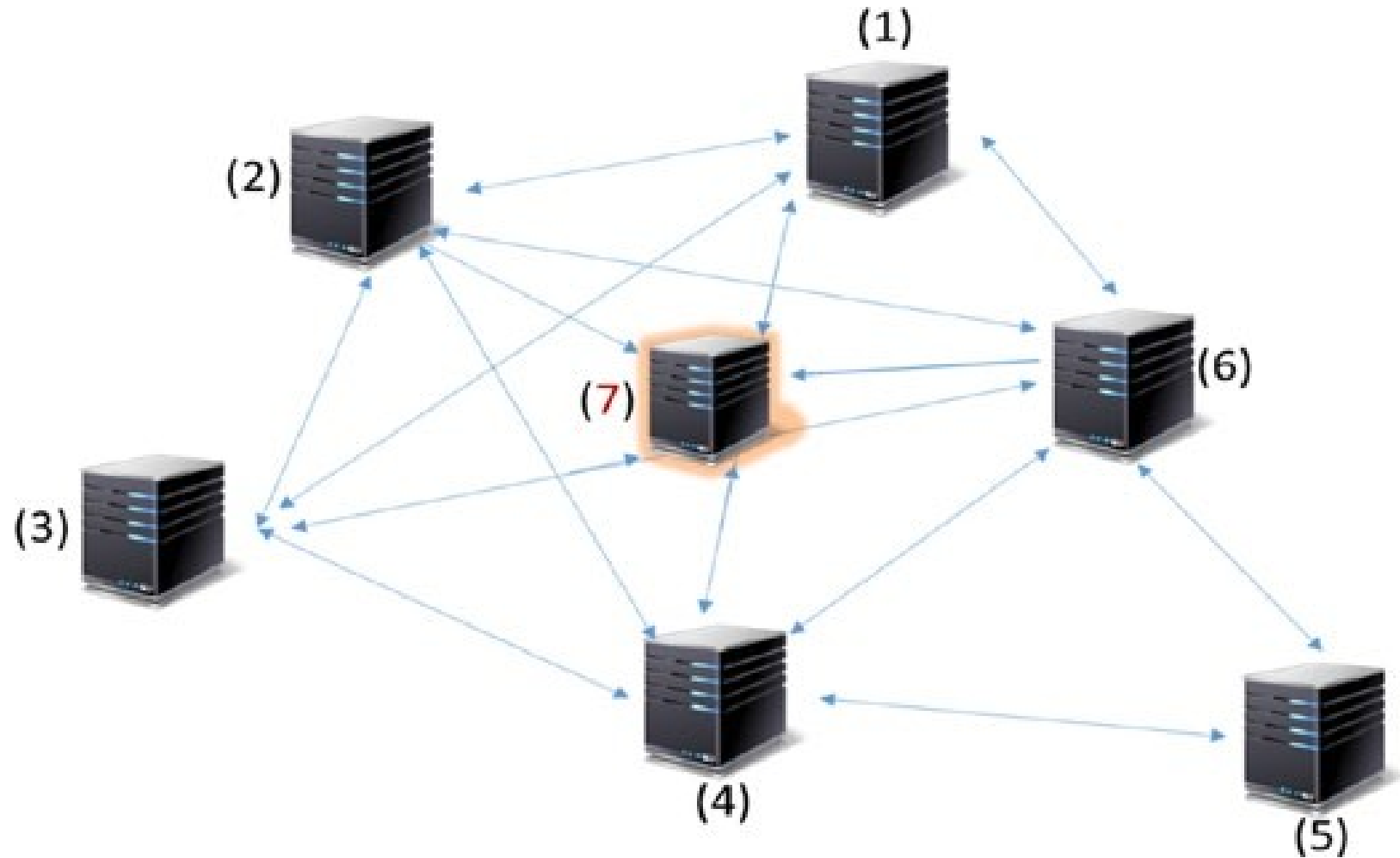


- To connect to a new peer,
 - the node establishes a TCP connection on port 8333
 - port 8333 is well known for Bitcoins but **could be different**
- Then the two nodes **handshake with information** such as
 - version number
 - Time
 - IP addresses
 - height of blockchain, etc.

- Step-4:
- The **requested nodes will respond** with
 - the list of IP addresses and corresponding port numbers of the other active Bitcoin nodes **that they are aware of**.
- Note that, **only one peer may be enough to bootstrap** the connection of a node to the Bitcoin network
- The node must continue to discover and connect to new peers.
 - This is because **nodes come and go** at will and no connection is reliable.



- Step-5:

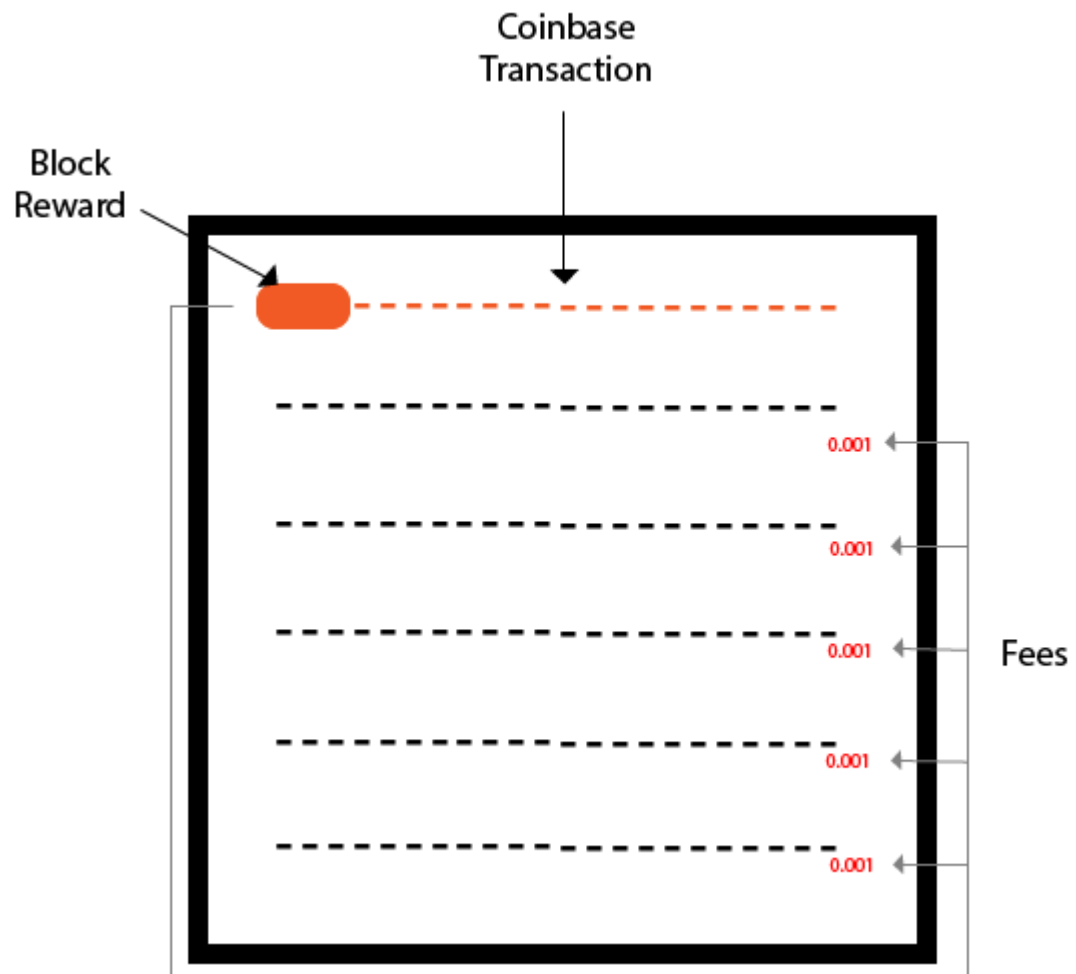


Bitcoin Transactions

Categories of Bitcoin transactions:

1. Coinbase transaction:

- A coinbase transaction is the **first transaction in a block**
- A newly generated bitcoin by bitcoin network for miners as a transaction fees
- A new generated bitcoin which is not spent anywhere (no UTXO).



2. Regular transactions:

- The regular transactions are very similar to currency exchanges.
- In Bitcoin, everything is present as transactions.
- You can create as many public addresses as you want.
- You can spend those transactions (unspent transactions or UTXOs) using your private key.

Gossip protocol:

- The receiver Bitcoin nodes further broadcast the transactions to other nodes
- and the entire network is usually flooded with transactions.
- Is called **gossip protocol**
- Gossip is the information relayed by each participant repeatedly to another member chosen at random and tells them all they know about the transaction
- It plays an important role in preventing double-spend attacks.

Bitcoin Transactions Revisited

- Bitcoin addresses are double-hashed output of the public key
 - Which is first hashed using SHA256
 - Then by RIPEMD160 hashing algorithms
 - Bitcoin addresses are derived from a 160-bit public key hash (RIPEMD-160).
 - This 160-bit hash is then Base58Check encoded, which results in an address that is 26 to 35 characters long (some can be 42 or 62 characters long).

Bitcoin Transactions Revisited

How They Work Together in Bitcoin Transactions

Step	Action
1	A user generates a private key (random 256-bit number).
2	From this private key, a public key is derived using elliptic curve multiplication.
3	The public key is hashed (SHA-256 + RIPEMD-160) to create a Bitcoin address .
4	To send Bitcoin , the sender signs the transaction using their private key (proving ownership).
5	The Bitcoin network verifies the signature using the sender's public key , ensuring the transaction is valid.

Example of a Bitcoin Transaction

Let's assume Alice wants to send **0.5 BTC** to Bob.

Transaction Breakdown

Component	Details
Input	Alice's previous UTXO of 1 BTC .
Output 1	Bob's address receives 0.5 BTC .
Output 2 (Change)	Alice gets back 0.49 BTC (after deducting a 0.01 BTC fee).
Digital Signature	Alice signs the transaction using her private key.
Broadcast	The signed transaction is sent to Bitcoin nodes.
Mining & Confirmation	Miners verify, include in a block, and confirm the transaction.



Scripts & Bitcoin Scripts

- A list of instructions recorded with each transaction
- It describes how the next person can gain access to those Bitcoins received and spend by them: e.g: sending funds from a wallet
- Bitcoin uses **stack-based, non-turing-complete** scripting language
 - evaluated from **left to right with limited functionality and cannot have loops.**
- **Turing complete language:** If the language has simple branching and a recursive syntax then it is Turing complete. e.g: C, C++, or Java

<Bob's signature> <Bob's public key> OP_DUP OP_HASH160 <Bob's public address>
OP_EQUALVERIFY OP_CHECKSIG.

Step 1:

<Bob's signature> <Bob's public key> OP_DUP OP_HASH160 <Bob's public address>
OP_EQUALVERIFY OP_CHECKSIG

Pushing <Bob's signature> on to the stack

<Bob's signature>

<Bob's signature> <Bob's public key> OP_DUP OP_HASH160 <Bob's public address>
OP_EQUALVERIFY OP_CHECKSIG

Followed by <Bob's public key>

<Bob's public key>
<Bob's signature>

Step 2:

<Bob's signature> <Bob's public key> OP_DUP OP_HASH160 <Bob's public address>
OP_EQUALVERIFY OP_CHECKSIG

Now OP_DUP comes into play which first pops <Bob's public key> out of the stack.

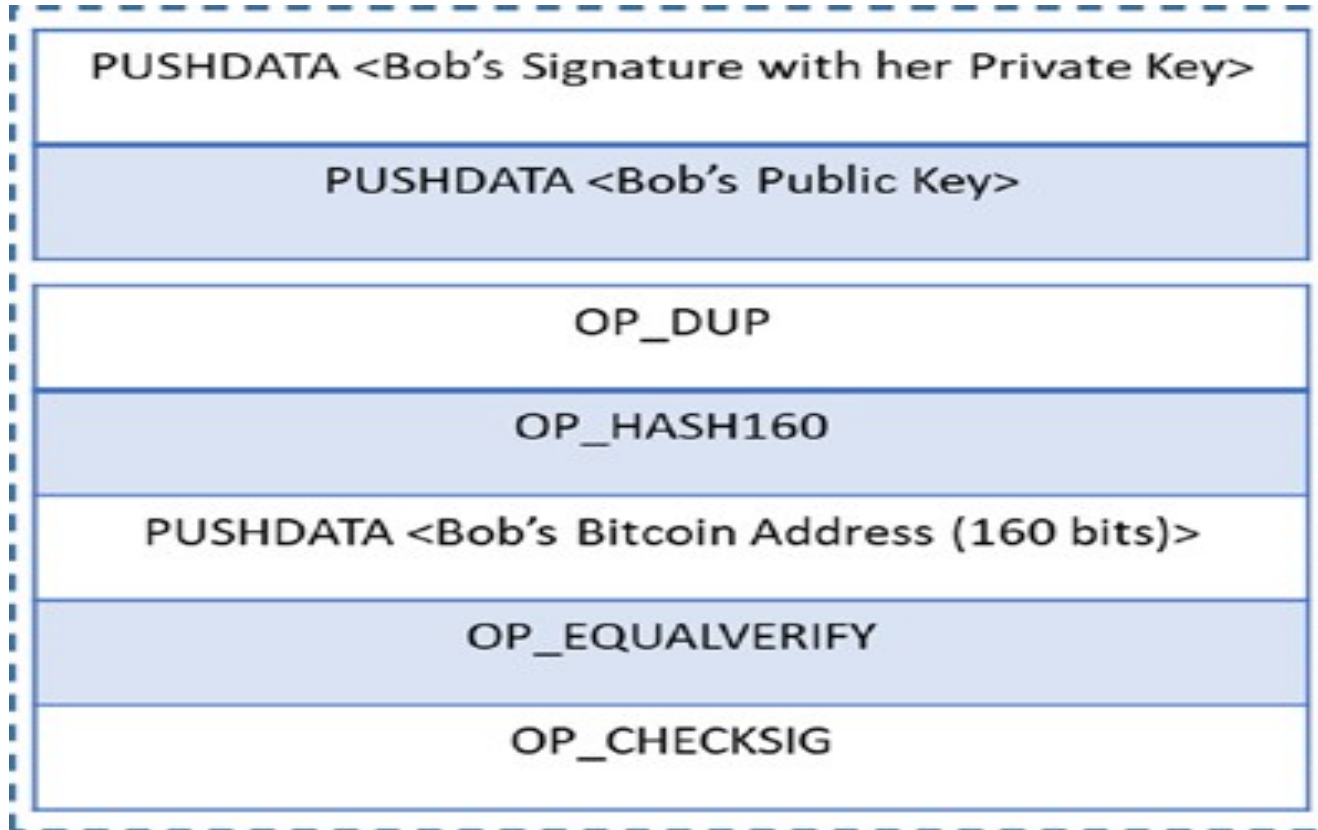
<Bob's signature>

Duplicates it, and then pushes both the original and the duplicate onto the stack

Bitcoin Scripts

- The execution time for scripts depend on the **number of instructions**.
- There are **256 instructions** in total. Out of that:
 - 166 are in use, 15 are disabled , 75 are kept reserved maybe for later usage.
- Script comprises of
 - mathematical, logical (if/then), error reporting, and just return statements.
 - **Some additional cryptographic** instructions such as hashing, signature verification, etc.
- Transaction Output = Amount of Bitcoins to transfer + **Output Script**
- Transaction Input = Reference to previous transaction output + **Input Script**

Instruction Sets



Combined Validation Script

Following are a few instruction sets fields:

- OP_DUP: It just duplicates the top item in the stack.
- OP_HASH160: It hashes twice, first with SHA256 and then RIPEMD160.
- OP_EQUALVERIFY: It returns TRUE if the inputs are matched, and FALSE otherwise and marks the transaction invalid.
- OP_CHECKSIG: Checks if the input signature is a valid signature using the input Public Key
- **To execute these instructions, we just have to push these instructions on to the stack and then execute.**

The corresponding stack-based implementation would be as follows (Figure 3-25).

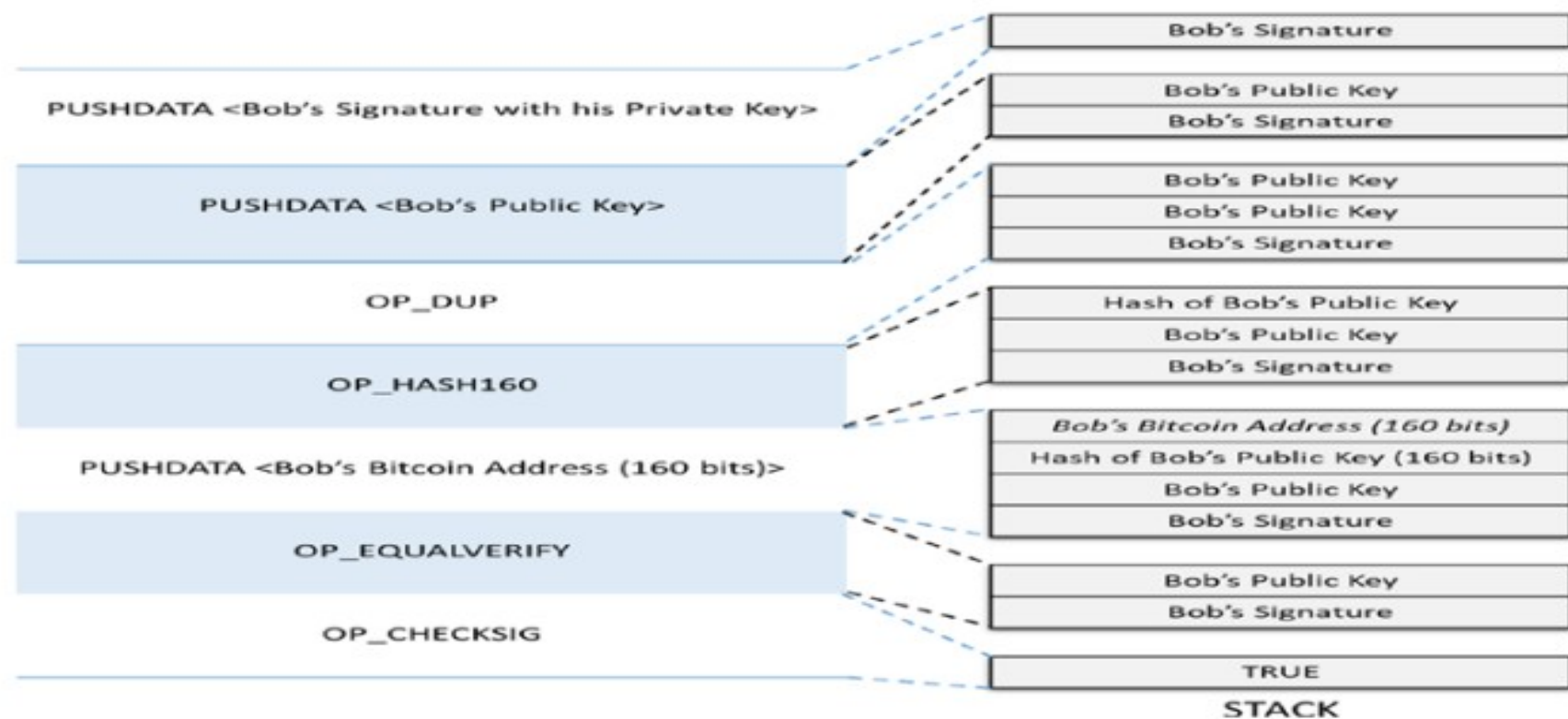


Figure 3-25. Example of stack-based implementation of Bitcoin script

(Confirmed on network, output is spent)

Tx_(n-1)

input

(Unlocking script for spent Tx (n-2) UTXOs)

Output

(Contained UTXOs that were spent by input (n))

(Confirmed on network, output is unspent)

Tx_(n)

input¹

(Unlocking script for spent Tx (n-1) UTXOs)

Output

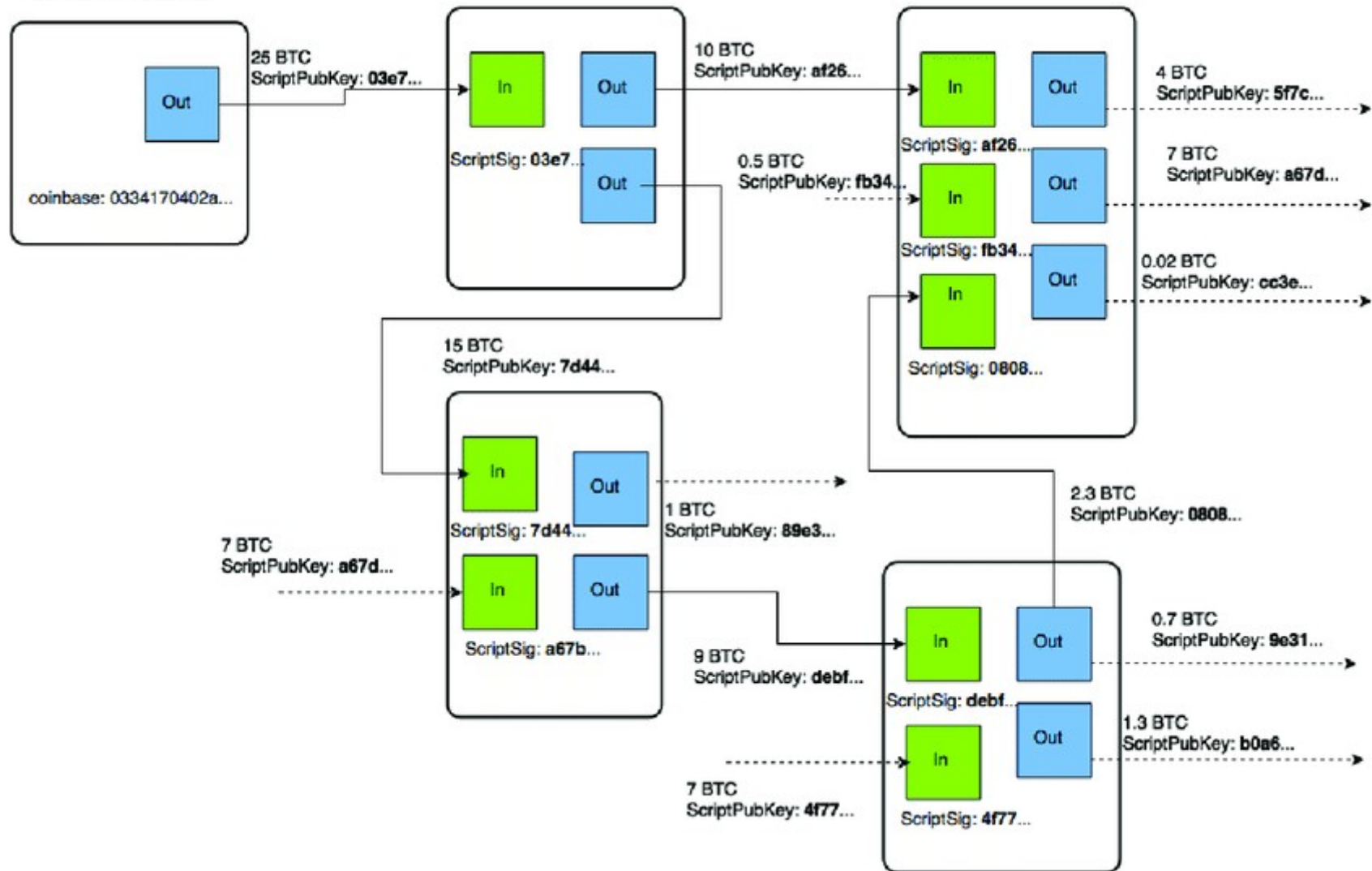
(Contained UTXOs awaiting input (n+1))



ScriptSigs- Input Script

- Is the mechanism of:
 - how to consume the previous transaction that you are trying to spend.
- **ScriptSig is unlocking script** , it has two components:
 - **the public key and the signature**
 - signature provided by ECDS Algo
- It proves the **ownership of the public key**
- ECDSA (Elliptic Curve Digital Signature Algorithm)

Generation Transaction



ScriptPubKey - Output Script

- It is input for future transaction
 - check the signature against the public key
- Output scripts = the public key of the recipient + the amount of Bitcoins being transferred
- Unless this output is spent, it remains a UTXO, waiting to be spent.
- **ScriptPubKey is used** -in the previous transaction to lock the transaction to the rightful owner
- These two scripts, **ScriptSig of current transaction and ScriptPubKey of previous transaction**, are combined and run.

Bitcoin Wallets

Bitcoin Wallets

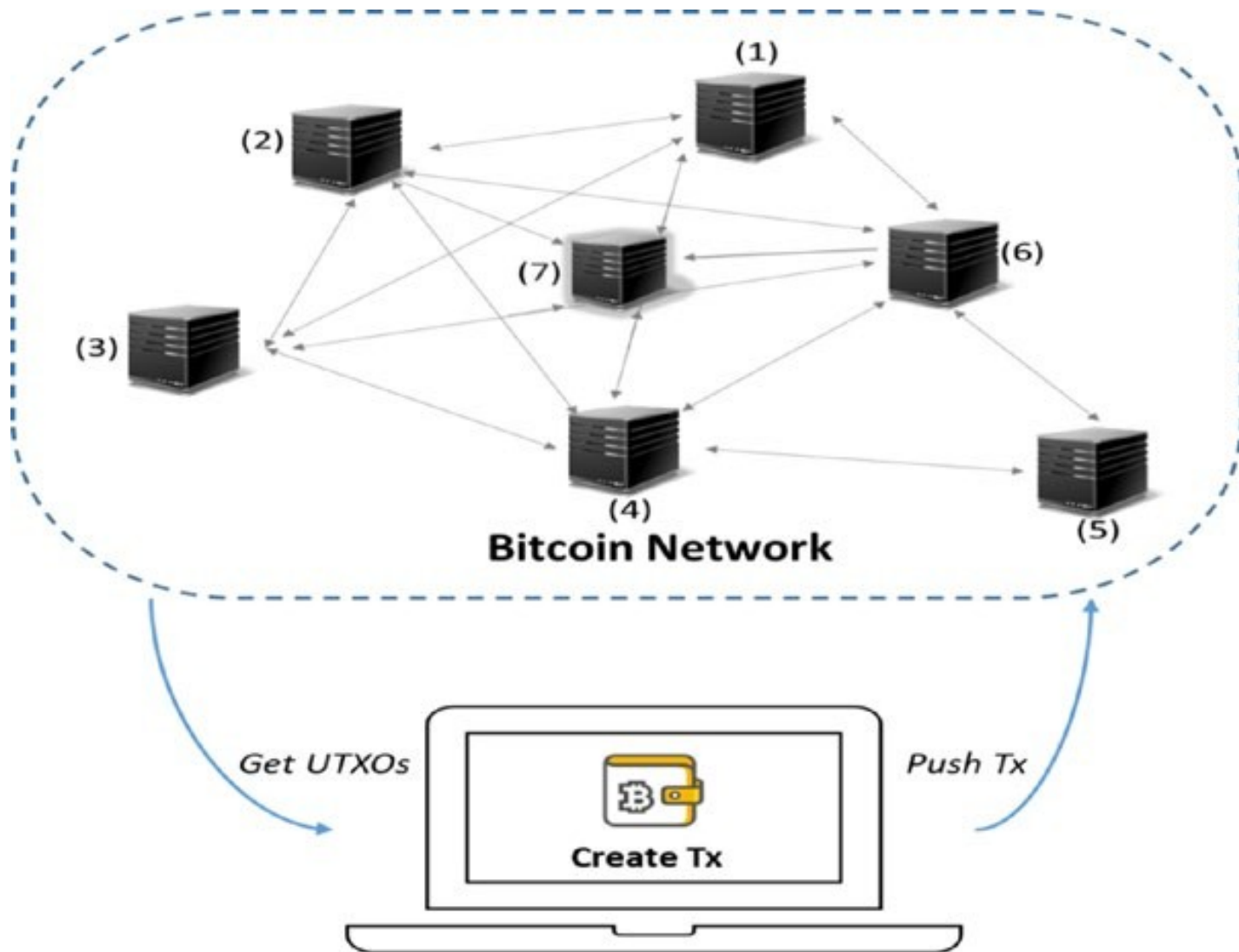
- Bitcoin wallets are very similar to the wallet you use in your daily life.
- In the Bitcoin setting, the accounts or wallets are represented by the **Bitcoin address**.
- If Alice wants to pay some amount to Bob, he must first generate a key pair (private/public keys).
- Bitcoin uses the ECDSA algorithm.

Bitcoin Wallets

- Bitcoin wallets are **nothing but the SPVs(simplified payment verification)** and are served by the full nodes.
- Wallet **saves your private/public** key pair and to be able to make and receive transactions.
- This can be done by creating your own wallet software or by using a third-party wallet service.



- **Be careful** with the wallet service providers because you are allowing them to take control of your private key.
- Whether they **deliberately take** your Bitcoins or they **themselves are hacked**
- At the end of the day, all wallet-service **providers are centralized**, though the decentralized Bitcoin network.



- An example of an SPV client that can serve as a Bitcoin wallet is “BitcoinJ.” **BitcoinJ is actually a library to work with**
 - the Bitcoin protocol
 - maintain a wallet
 - send/receive transactions without needing a local copy of Bitcoin Core
 - initiate/validate transactions.
- It does not require a full node such as a Bitcoin Core node
- It can function as a thin client node.
- Though it is implemented in Java, it can be used from any Compatible language such as JavaScript and Python.